

Project 5 - Analysing Adversarial Attacks on Network Data Classifiers

ANTONIO CRUDELE, Politecnico di Torino, Italia
MEGI DOKOLLARI, Politecnico di Torino, Italia
GIOVANNI FRISENDA, Politecnico di Torino, Italia
CALOGERO MARIA, Politecnico di Torino, Italia

ACM Reference Format:
Antonio Crudele, Megi Dokollari, Giovanni Frisenda, and Calogero Maria. 2025. Project 5 - Analysing Adversarial Attacks on Network Data Classifiers. 1, 1 (January 2025), 19 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

CONTENTS

Contents	1
1 Introduction	2
2 Data Acquisition and Exploration	2
2.1 Data Acquisition	2
2.2 Data Preprocessing	2
2.3 Exploratory Data Analysis	3
2.4 Data Visualization	5
2.5 Unsupervised Data Analysis	7
3 Supervised Data Analysis	10
3.1 Classifier Selection and Cross Validation	10
3.2 Classifier Evaluation	11
4 Adversarial Attack	14
4.1 Random Noise	14
4.2 Feature Specific Noise	14
4.3 Adversarial Attack with ART	15
4.4 Countermeasure Exploration	17
5 Conclusion	18
References	19

Authors' Contact Information: Antonio Crudele, s338541@studenti.polito.it, Politecnico di Torino, Torino, Italia; Megi Dokollari, s346293@studenti.polito.it, Politecnico di Torino, Torino, Italia; Giovanni Frisenda, s337127@studenti.polito.it, Politecnico di Torino, Torino, Italia; Calogero Maria, s297844@studenti.polito.it, Politecnico di Torino, Torino, Italia.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2025/1-ART
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Ensuring the security of Machine Learning (ML) classifiers is essential for protecting systems against cyberattacks and fraud. Adversarial attacks, which generate subtle modifications to input data, can mislead models into making incorrect predictions. Although these attacks are well studied in image classification, they also pose significant threats to tabular data, such as network traffic.

In this project, we investigate the impact of adversarial attacks on classifiers for application detection using network traffic data. This task, crucial for network security and traffic management, highlights the vulnerabilities of ML models and the importance of developing robust defenses against these sophisticated threats.

2 Data Acquisition and Exploration

This section provides an overview of the initial steps undertaken to prepare the dataset for analysis. It begins with data acquisition, followed by preprocessing techniques to clean and format the data appropriately. An exploratory data analysis was then conducted to uncover key patterns and relationships within the dataset. Data visualization techniques were employed to further illustrate these insights effectively, while unsupervised learning methods were utilized to analyze the data structure and extract meaningful clusters or trends.

2.1 Data Acquisition

First of all, the *IP Network Traffic Flows Labeled with 75 Apps* dataset is loaded into a pandas dataframe. The dataframe is then filtered to include only the traffic related to the following applications: Youtube, Amazon, Gmail, Windows Update, Skype, Facebook, Dropbox, Yahoo, Twitter, and MSN.

These ten applications will represent the classes considered in our project.

2.2 Data Preprocessing

Before proceeding, we preprocess the dataset to make it more suitable for the model. We remove redundant features, such as 'Flow.ID' and 'L7Protocol', which are already represented by other features, and exclude features with a single constant value, as they provide no discriminative power. The removed features include 'Bwd.PSH.Flags', 'Fwd.URG.Flags', 'Bwd.URG.Flags', 'CWE.Flag.Count', 'Fwd.Avg.Bytes.Bulk', 'Fwd.Avg.Packets.Bulk', 'Fwd.Avg.Bulk.Rate', 'Bwd.Avg.Bytes.Bulk', 'Bwd.Avg.Packets.Bulk', 'Bwd.Avg.Bulk.Rate', and 'Label'.

We then convert certain features for better model compatibility. Specifically, 'Source.IP' and 'Destination.IP' are transformed from strings to integers, allowing the model to treat them as numerical inputs. Additionally, we apply one-hot encoding to the 'Protocol' feature, converting it into binary columns for better interpretability by the model. The 'Timestamp' feature is also split into granular components (year, month, day, hour, minute, second) and then converted into a single numerical value representing the seconds since January 1, 1970.

Through these transformations, the dataset now includes refined features that better align with the requirements of our Machine Learning approach, improving the model's capacity to interpret and learn from the data.

As a result, the final dataset consists of 471,184 samples and 81 features.

Next, we prepare the data for training and testing by defining the target variable y with one-hot encoding for 'ProtocolName' and extracting the features X . We split the data into training and test sets with an 80/20 ratio, maintaining class distribution via stratification.

To ensure consistent scaling, we apply the `MinMaxScaler` to normalize the feature values within the $[0,1]$ range: this step ensures no feature dominates the model due to differing magnitudes. The scaler was fitted on the training data to avoid data leakage and subsequently applied to both the training and test sets.

Finally, we convert the scaled data and target variable into pandas DataFrames for easier manipulation and to retain feature names, completing the preprocessing phase with a clean and structured dataset ready for modeling.

- Destination.Port:** Similarly to the Source Port boxplot, we can see that Dropbox has the largest IRQ, while the others have much narrower distributions. This likely means that Dropbox uses multiple protocols and dynamic ports to facilitate file synchronization and data transfers. In this boxplot the outliers are mostly located in the upper end of the range and the median values are near zero.
- Flow.IAT.Std (Inter-Arrival Time Standard Deviation):** Dropbox shows the largest IQR and higher median compared to other applications. This might be because Dropbox experiences irregular flow timings due to the nature of file synchronization, where data transfers occur in bursts. Other applications have much smaller IQRs and lower medians, indicating more consistent inter-arrival times, which is typical of real-time and streaming applications. All applications have outliers, which could be the result from network congestion, retries, or less frequent data exchanges.
- Bwd.IAT.Std (Backward Inter-Arrival Time Standard Deviation):** Dropbox has the widest IQR and the highest median, while others are narrower and with lower medians. Applications with low variability, such as Twitter and Gmail, can likely be optimized for consistent performance, while high variability applications like Dropbox might benefit from adaptive resource allocation to handle bursts efficiently. Outliers are present for all applications, particularly in Amazon, Skype, and Dropbox.
- Init_Win_bytes_forward (Initial Window Size (bytes) for forward traffic in a TCP connection):** Most applications have similar distributions, with medians around the lower half of the scale. Dropbox is the application with the smallest IRQ and lowest median. All applications have a large number of outliers.
- Idle.Min (Minimum Idle Time):** Some applications like Dropbox, Skype and Gmail have wider IQRs, while most of them have very narrow ones. All categories except for Dropbox have a lot of outliers.

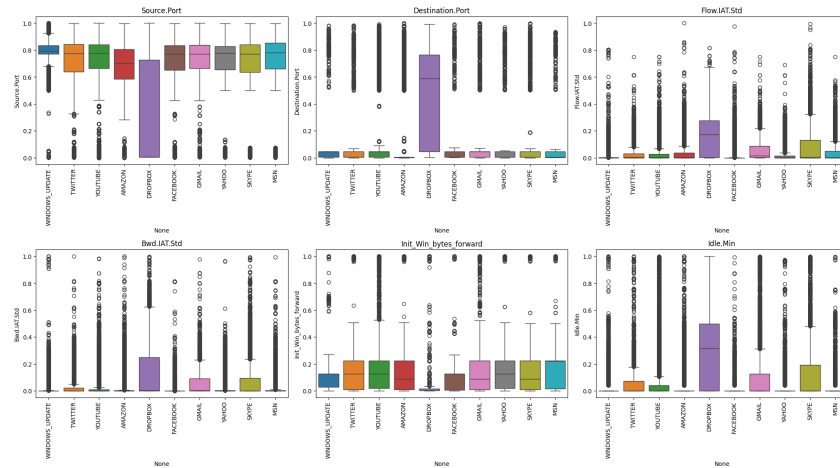


Fig. 2. Box Plots

2.3.3 Histograms. Another visualization technique are histograms [2], which are useful to describe the frequency of numerical values. Like for the boxplots, only the most interesting features are plotted (Figure 3).

- **Source.Port and Destination.Port:** We observe concentrations between 0.6-0.8 (Source.Port) and 0-0.2 (Destination.Port). This probably reflects that some applications frequently associate with well-defined ports. For example: Source.Port exhibits a fairly diverse pattern, with applications such as WINDOWS_UPDATE and YOUTUBE visible in wider ranges. Destination.Port is much more concentrated toward low values, indicating that the destination ports for many applications are narrow.
- **Flow.IAT.Std:** The Inter-Arrival Time (IAT) standard deviation of the flow variable has a very concentrated distribution toward low values. This is typical for traffic that maintains regular flows with little temporal variation.
- **Bwd.IAT.Std:** Similar behavior to Flow.IAT.Std, with most values close to 0. This suggests fairly stable two-way traffic.
- **Init_Win_bytes_forward:** This feature shows a wider distribution, with relevant values concentrated in both the low end (0-0.2) and high end (0.8-1.0). The high end might indicate applications that start streams with large window sizes, such as WINDOWS_UPDATE or YOUTUBE.
- **Idle.Min:** Lower values indicate very short periods of inactivity. These are common in applications with near-continuous connections (e.g., streaming, upgrades), where traffic maintains a steady pace.

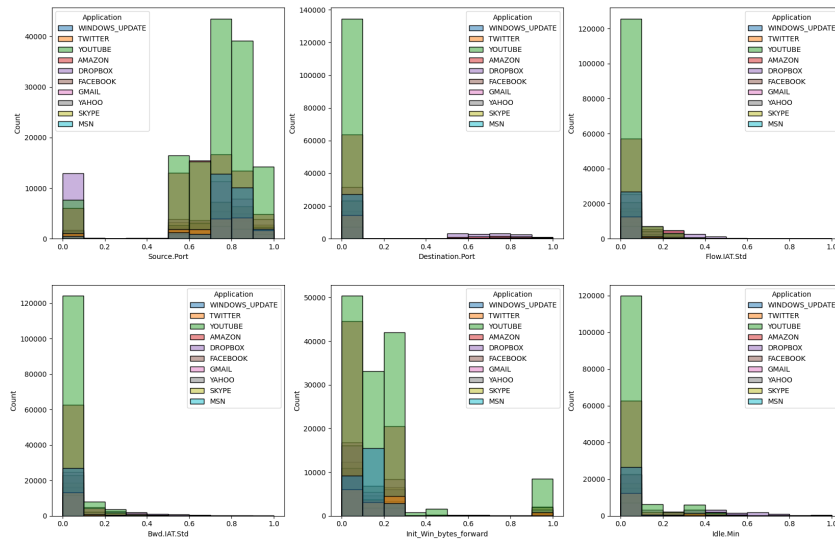


Fig. 3. Histograms

2.4 Data Visualization

We apply Principal Component Analysis (PCA) [3] to reduce the dimensionality of the data, eliminating redundancy and improving efficiency. By projecting the data into lower-dimensional spaces, PCA aids visualization and helps retain the most relevant features, reducing noise and simplifying the dataset. This results in faster model training and better overall performance.

From the PCA, we selected 21 features that collectively accounted for an exceptionally high percentage of the dataset's variance illustrated in Figure 4.

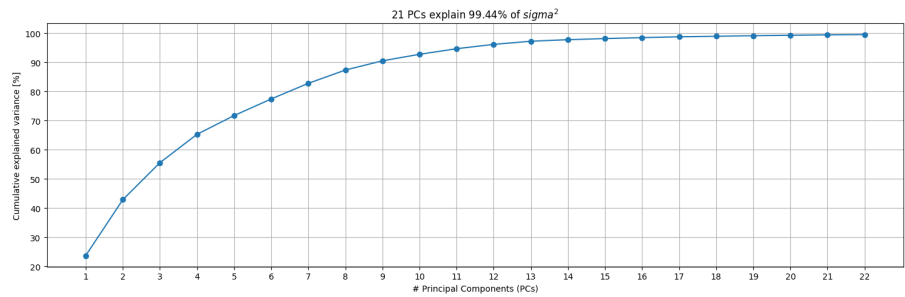


Fig. 4. PCA Visualization

So we initialize the PCA with these 21 components and then compute and visualize the loading score, which shows how strongly the components describes the original features.

This enabled us to obtain a highly accurate representation of the data, capturing its most significant characteristics. Next, by visualizing the first three principal components in a three-dimensional space in Figure 5, we were able to observe how these components were vectorially constructed from the original features.

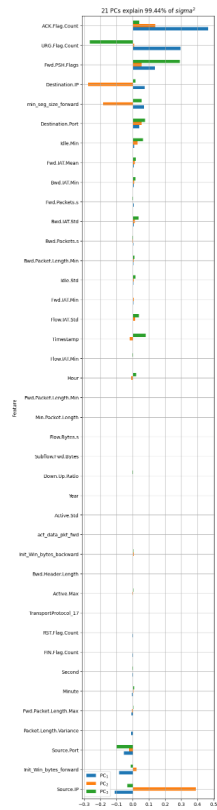


Fig. 5. Three dimensional representation of features

Using this insight, we identified features that had no meaningful contribution to the formation of this reduced-dimensional space.

These features have little to no impact on the overall explanation of the dataset's variability, as indicated by their minimal or zero contribution in the plot. As a result of our analysis, we determined that only the feature 'Year' was irrelevant and could be safely removed.

2.5 Unsupervised Data Analysis

As a first step we scale the whole dataset, since for clustering there is no need to divide it into train, validation and test sets, and then we remove the unnecessary features previously found. Then, we take 10000 random samples in order to make the computation feasible.

In order to visualize the clusters, we reduce the feature dimensionality with a 2-component t-SNE [4], which resulted in better visualization than the PCA.

2.5.1 K-Means. This algorithm is ideal for spherical clusters and noise-free datasets. It's also simple and fast for well separated data.

First, we find the best value for the hyperparameter k , which indicates the number of clusters, and for that we use the Elbow Method. Figure 6 shows the k-Means clustering error and the silhouette score. We choose $k=13$.

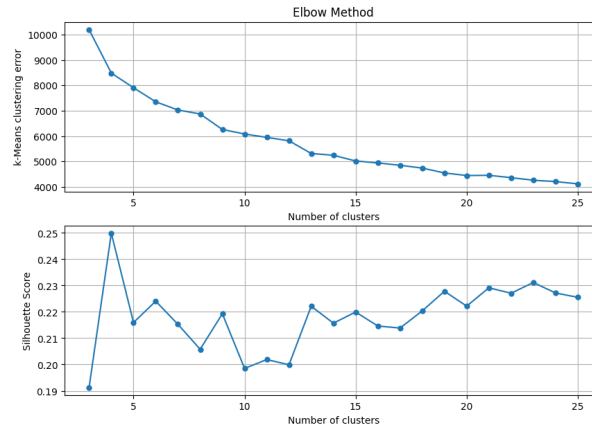


Fig. 6. Elbow Method for k-Means

After this, we tune the other hyperparameters `init`, `n_init` and `max_iter`, obtaining `init: 'k-means++'`, `max_iter: 50`, `n_init: 18`.

With the obtained results, we execute the K-Means algorithm and compute the metrics.

We obtain a Silhouette Score equals to 0.23, which is relatively low, indicating that the clusters are not clearly separated and that there may be some degree of overlap between the groups. In addition, an Adjusted Rand Index (ARI) equals to 0.06 indicates that the clustering obtained is not very similar to the ground truth. However, we still obtained a good value of the Rand Index (RI) of 0.77.

We then compare the tuned K-Means results with the one that has $k=10$, which is the number of our labels, and the Ground Truth (Figure 7).

2.5.2 GMM (Gaussian Mixture Model). This algorithm is useful when clusters have nonspherical shapes or overlap. It allows for probabilistic modeling.

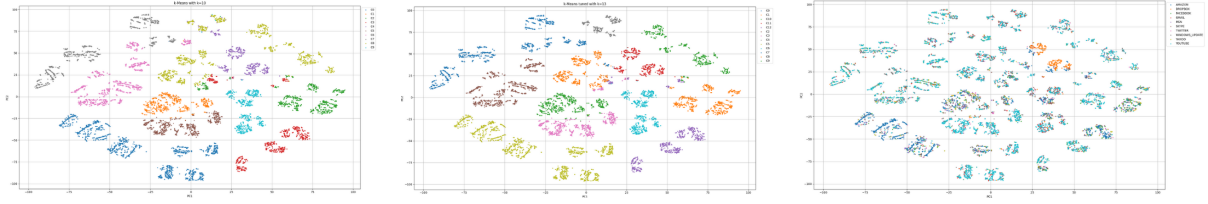


Fig. 7. Clustering using k-Means

Similarly to the K-Means, we tune the hyperparameter k that refers to the number of clusters using the Elbow Method (Figure 8), this time plotting the GMM total log-likelihood score and the silhouette score. Again, we choose $k=13$.

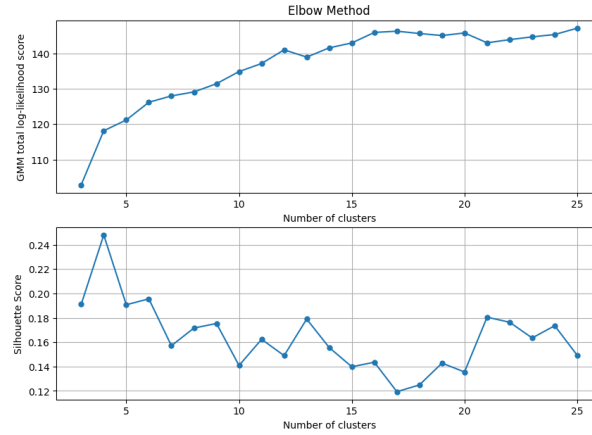


Fig. 8. Elbow Method for GMM

Then we also tune the other hyperparameters `init_params`, `covariance_type`, `tol` and `max_iter`, obtaining `covariance_type: 'spherical'`, `init_params: 'kmeans'`, `max_iter: 50`, `tol: 0.001`.

After computing the metrics with the tuned hyperparameters, we can see that the results are very similar to those obtained through k-Means, so the same considerations made above apply.

Lastly, we compare the obtained GMM with the one having $k=10$ and with the Ground Truth (Figure 9).

2.5.3 DBSCAN. Good for identifying clusters of irregular shapes and handling noise. It's ideal for data with variable density.

For DBSCAN, we performed several simulations by changing the `eps` and `min_samples` parameters. In the end, we considered the combination of parameters that maximizes the silhouette score, as we can see in Figure 10.

Then, we execute DBSCAN with tuned hyperparameters and calculate various metrics. The number of clusters obtained (including noise) is 21 with the following sizes: 114, 1385, 3123, 2277, 427, 520, 64, 280, 706, 230, 81, 618, 42, 33, 12, 26, 20, 6, 19, 12, 5. The model found some clusters with very small sizes (e.g., with less than 10 points), which could indicate that it identified “anomalies” or noise in the data.

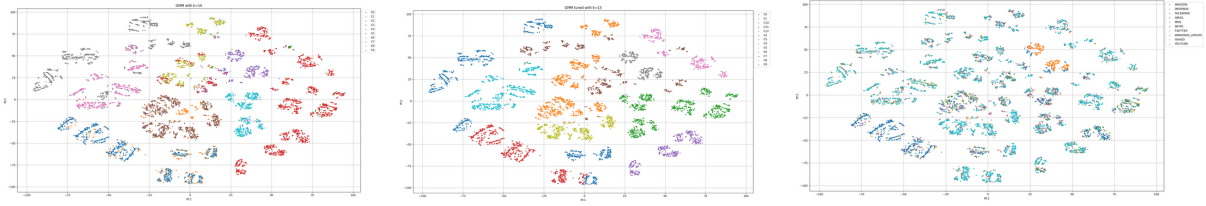


Fig. 9. Clustering using GMM

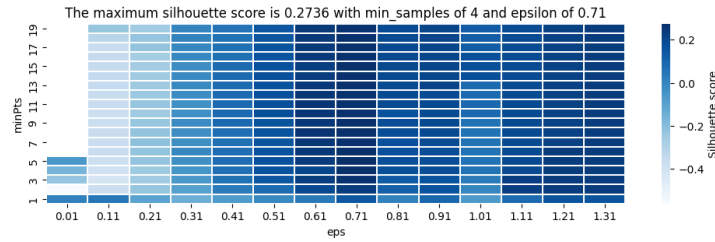


Fig. 10. Silhouette score grid

The other metrics are very similar to the previous algorithms, except that the silhouette score value improves slightly. Despite the improvement over k-Means and GMM in terms of silhouette, the silhouette value remains relatively low, indicating that the clusters are not completely separated and that there may be a lot of overlap between the points.

Lastly, we visualize the clusters in Figure 11, comparing them with the Ground Truth.

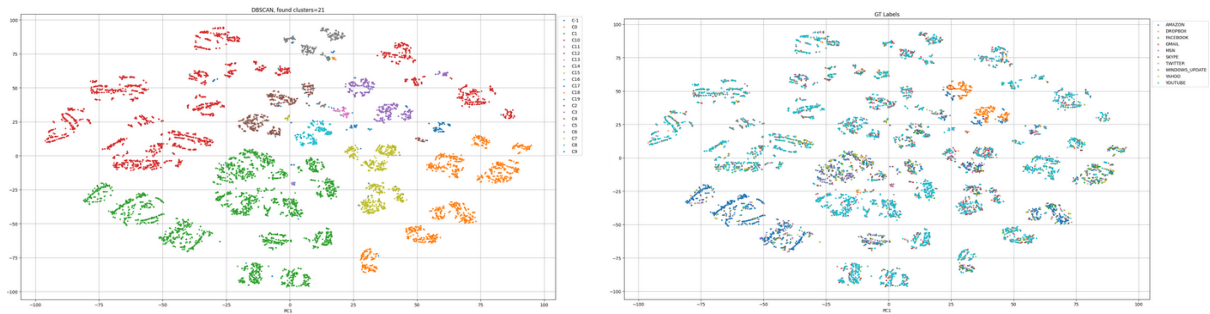


Fig. 11. Clusters using DBSCAN

In conclusion, the poor results obtained for all the algorithms may be due to the fact that the 10000 samples considered are not representative enough of the entire dataset. However, the sampling was necessary because of our limitations in computational resources. Even using stratified sampling, the performance does not change significantly.

3 Supervised Data Analysis

This section focuses on the application of supervised learning techniques to classify the data and evaluate model performance. It begins with the selection of classifiers, exploring the suitability of Random Forest, Decision Tree, and Neural Network models for the given dataset. A cross-validation approach was employed to ensure robust and unbiased model evaluation. Finally, the evaluation of the best-performing classifier was conducted, providing valuable insights into its effectiveness in addressing the classification task.

3.1 Classifier Selection and Cross Validation

3.1.1 Random Forest Classifier. As first classifier we use the Random Forest, and we performed a k-fold cross validation to evaluate the best value for the hyperparameter `max_depth` [5].

In particular, we evaluate the values ranging from 15 to 20 (excluded) using a 5 fold cross validation, and then we plot the results in a validation curve.

As we can see in Figure 12, the accuracy of both the training and validation set increases with the increase of the hyperparameter `max_depth`, and the gap between the two curves also slowly increases, which means that higher values for this hyperparameter might result in overfitting.

So to avoid overfitting, we chose `max_depth = 18` as the best hyperparameter value, which gives approximately a training set accuracy score of 0.849 and a validation set accuracy score of 0.788.

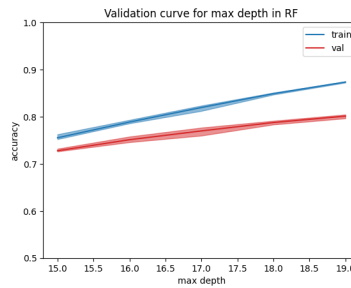


Fig. 12. Random Forest Validation Curve

3.1.2 Decision Tree Classifier. About the Decision Tree Classifier [5] we perform a comprehensive hyperparameter tuning process to enhance its performance and generalization ability.

Including first, the criterion, which determines how the quality of splits is measured, was explored using two options: 'gini' and 'entropy'. Second, the maximum depth of the tree, which directly influences the complexity of the model and its ability to capture patterns in the data, was optimized to ensure a balance between performance and overfitting. Finally, the minimum number of samples required to split a node was adjusted to control how the tree grows and to prevent unnecessary splits, ensuring the model remains both effective and interpretable.

To identify the optimal configuration, a grid search was performed over 20 different combinations of these parameters. For each combination, a 5-fold cross-validation was conducted, ensuring that the data was split into training and validation sets in a robust and unbiased manner. This approach enabled a thorough exploration of the parameter space and provided reliable performance metrics.

The accuracy of the model was calculated for both the training and validation sets across all folds. Based on these results, the combination yielding the highest mean accuracy on the validation set was selected as the best configuration.

The optimal hyperparameters identified were: `criterion = 'entropy'`, `max_depth = 19`, and `min_samples_split = 2`. This configuration achieved a cross-validated accuracy of 0.883, while the training set accuracy was slightly higher at 0.930.

This systematic tuning process ensured that the model is not only well-suited to the training data but also generalizes effectively to unseen data, minimizing the risks of overfitting or underfitting.

3.1.3 Neural Networks. Regarding the classification with Neural Networks (NN), the first step was to split the dataset into a training set and a validation set with an 80/20 ratio, maintaining class distribution via stratification. The data was converted into tensors to use with PyTorch, as described in [6].

We defined our neural network model with an architecture consisting of two hidden layers. The model takes as input the features of each sample, where the first hidden layer has 64 neurons, and the second hidden layer contains 32 neurons. Finally, the output layer consists of 10 units, corresponding to the 10 classes to be predicted.

For the activation function, we used `Tanh()` to introduce non-linearity, while in the forward pass of the network the input is passed through the defined layers of the network sequentially, and the output is computed accordingly. The loss function employed was the weighted cross-entropy loss, where weights were assigned based on the number of samples in each class. This approach ensured that the model did not become overly specialized in favor of any particular class.

As the first step, we performed cross-validation using the following parameters:

```
k_fold = 5, batch_size = 64, learning_rate = 0.001, num_epoch = 10
```

This approach allowed us to evaluate the model's performance across multiple subsets of the dataset and assess its generalization ability.

The results were consistent across all the folds, with a Cross-Validation Result of 0.63 ± 0.01 . This suggests that the model's performance is stable across different splits of the data, indicating that it is not overfitting or underfitting on specific subsets, and it can generalize well to unseen data.

Next, we focused on performing a grid search to find the best hyperparameters for the neural network. The hyperparameters we tuned included are:

```
learning_rate = [0.01, 0.001, 0.0001], batch_size = [64, 128, 256]
```

These parameters were evaluated over a fixed number of epochs set to 10 to maintain a reasonable computational time. This grid search allowed us to identify the most effective combination of hyperparameters for optimizing the neural network's performance.

The result depicted in Figure 13 suggests that the highest accuracy was obtained with the following values: `learning_rate = 0.01`, `batch_size = 128`, resulting in an accuracy on the validation set of 0.662%. The results showed that as the learning rate decreased, the accuracy worsened progressively. Additionally, reducing the learning rate and using a larger batch size led to further performance degradation. This suggests that the model is sensitive to both the learning rate and the batch size, and that smaller learning rates, combined with larger batch sizes, might hinder the model's ability to converge effectively.

After obtaining these results, we retrained the model with the selected parameters and iterated the training process until the accuracy no longer improved for two consecutive epochs. This approach was used to reduce computational costs and avoid overfitting. Subsequently, we generated plots depicting the variation of the loss and accuracy for both the training and validation sets, as presented in Figure 14a. Finally, we also visualized the confusion matrix of the resulting model to assess its performance in terms of class predictions in Figure 14b.

3.2 Classifier Evaluation

After performing a cross-validation for the Random Forest, Decision Tree and Neural Network classifiers, we take the best performing one, which is the Decision Tree, and we evaluate the results on the test set.

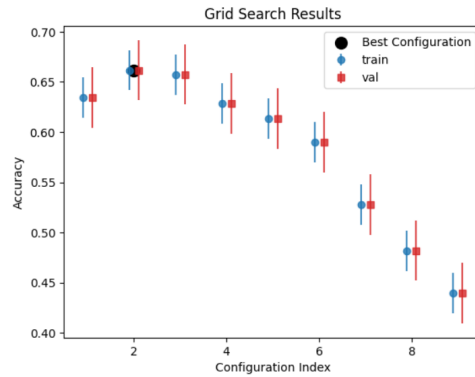


Fig. 13. Grid Search of Hyper Parameters of NN

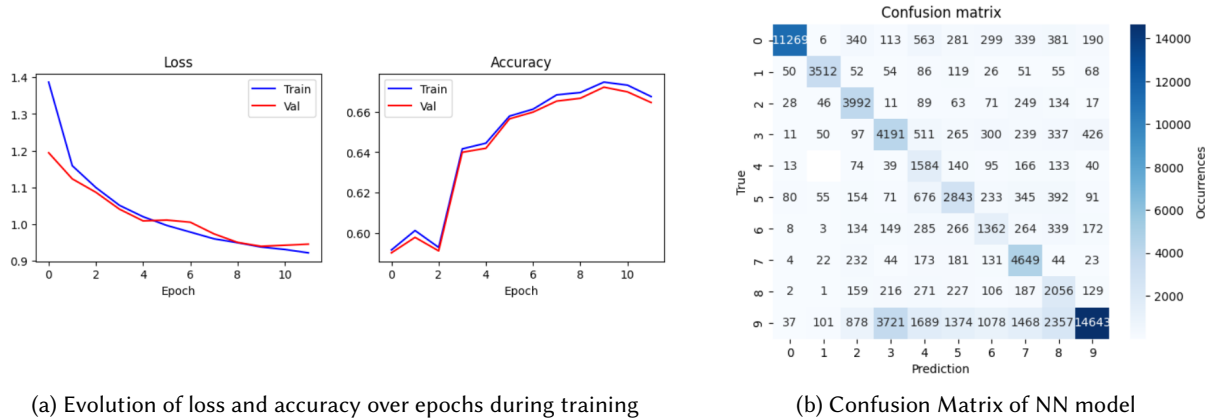


Fig. 14. NN optimized model performance

We get similar results to the validation set, with an accuracy value of 0.88, which means that most of the predictions are correct, as we can see in Figure 15.

ROC curve. The ROC curves (Receiver Operating Characteristic curves) are graphical tools used to evaluate the performance of a binary classifier. On the X-axis, the **False Positive Rate** (FPR) is represented, while on the Y-axis, the **True Positive Rate** (TPR) is displayed.

Information Extracted from the images are:

- **Shape of the Curves:**
 - The closer the curve is to the top-left corner (low FPR, high TPR), the better the model's ability to distinguish between classes.
 - Nearly diagonal curves represent performances close to random chance.
- **AUC (Area Under the Curve):**
 - The area under the curve (AUC) is an aggregated measure of the model's performance across all thresholds.
 - AUC values close to 1 indicate excellent predictive ability.
 - Values near 0.5 indicate random performance.

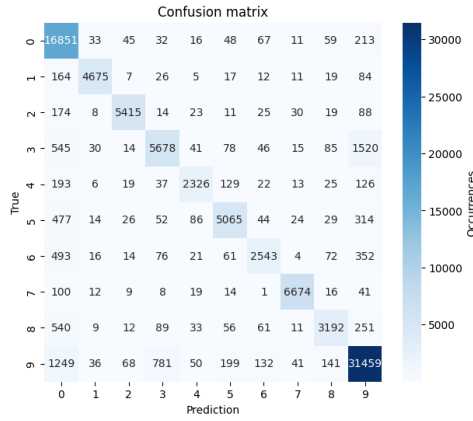


Fig. 15. Confusion matrix of Decision Tree

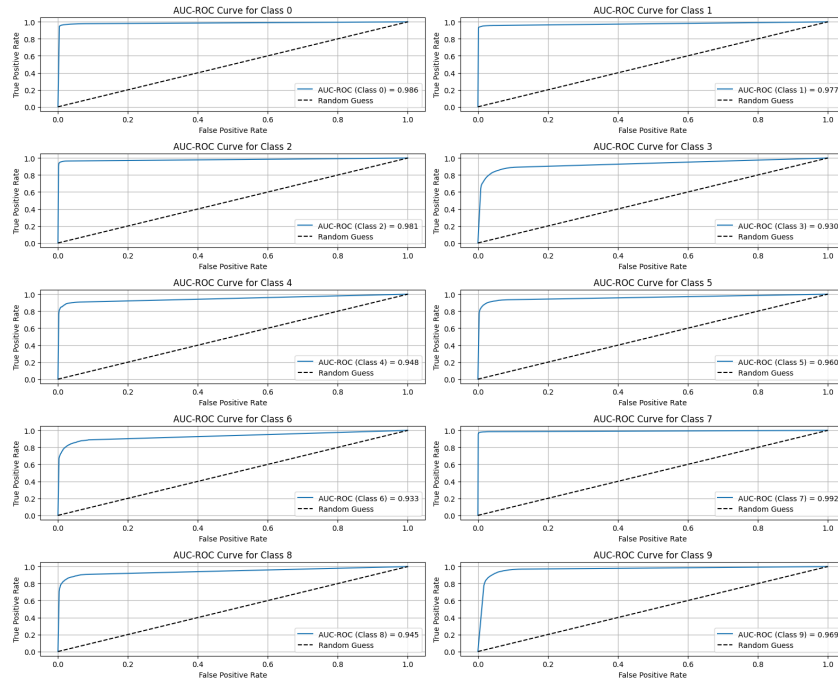


Fig. 16. ROC curve

Analysis of the Results. The analysis of the generated ROC curves shows that:

- Each graph represents the ROC curve for a specific class (*Class 0, Class 1, ..., Class 9*).
- The reported AUC values are very high for all classes. For example:
 - *Class 0*: AUC = 0.986
 - *Class 1*: AUC = 0.977

– ...

- The model's performance is excellent, with all AUC values above 0.93.

The model demonstrated excellent discriminative ability between classes, with very high AUC values for all categories. The proximity of the curves to the top-left corner in the ROC graphs and the high AUC values indicate that the binary classifier is highly effective.

4 Adversarial Attack

In this section, we explore the concept of adversarial attacks, which aim to manipulate machine learning models by introducing small but purposeful changes to the input data, leading to incorrect predictions. We begin by discussing various types of noise used in adversarial attacks, including random noise and feature-specific noise. Next, we introduce more sophisticated methods for generating adversarial examples using the ART library and focusing on the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD), both of which are well-known techniques within the field. Finally, we explore potential countermeasures to defend against these attacks, discussing strategies to enhance the model's robustness and mitigate the impact of adversarial inputs.

4.1 Random Noise

Now we apply random noise to the test data in order to measure the Robust Accuracy metric of the Neural Network Classifier.

First of all, we define a function that adds noise to the data and then we call this function on the test set using different values of noise. After that we evaluate the classifier on the noisy data and plot the degradation of the accuracy with respect to the noise intensity.

As we can see in Figure 17, the performance of the classifier drastically decreases as the noise increases.

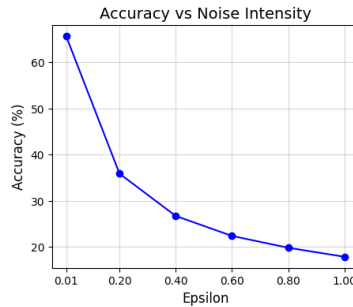


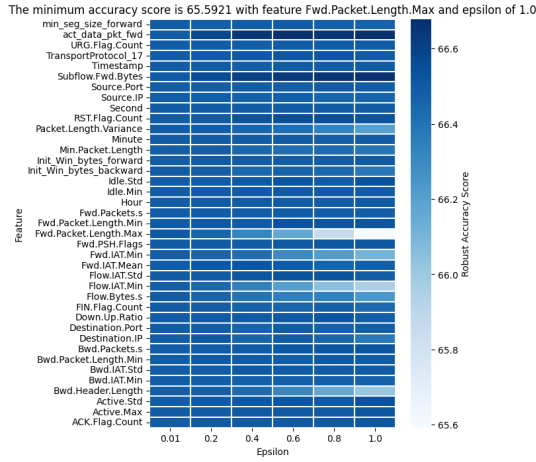
Fig. 17. Robust Accuracy of Neural Network Classifier

4.2 Feature Specific Noise

Now we want to add noise individually on each feature in order to find the most vulnerable ones. To do this, we define a function that adds noise to a specific feature, and then we call this function for each feature of the dataframe using different noise values.

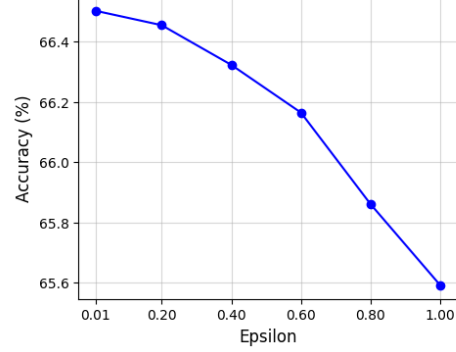
The results in Figure 18a show us that the Neural Network classifier is more sensitive to certain features, in particular the most vulnerable one is the `Fwd.Packet.Length.Max`, that leads to the worst Robust Accuracy.

In Figure 18b we can see how the Robust Accuracy decreases while increasing the noise intensity on `Fwd.Packet.Length.Max`. For example, at $\epsilon = 1.0$, the Robust Accuracy decreases by approximately 1%.



(a) Features sensitivity to noise

Accuracy vs Noise Intensity on Fwd.Packet.Length.Max



(b) Noise effect on Fwd.Packet.Length.Max

Fig. 18. Most vulnerable feature

Since at $\epsilon=0.8$ the Robust Accuracy drops by about 0.8%, we can say that this is the minimum value to have a significant impact on the Neural Network classifier.

These results are important to identify security issues and fix them, as an attacker can exploit these vulnerabilities to mislead the classifier while maintaining credible and realistic patterns. In particular, the attacker might focus on the most vulnerable features, requiring less effort while still successfully deceiving the classifier.

4.3 Adversarial Attack with ART

ART (Adversarial Robustness Toolbox) [7] is a Python library designed for Machine Learning security. It provides various tools for the creation and evaluation of adversarial attacks, which can be used to test the robustness of machine learning models against malicious inputs. In our work, we employed some tools from ART to simulate adversarial attacks on our neural network classifier [8], with the aim of evaluating the potential impacts these attacks might have on its performance. This allowed us to assess the model's vulnerability to perturbations and understand its behavior under adversarial conditions.

We considered a variable epsilon value, defined as `epsilon_values = [0.01, 0.05, 0.1, 0.2, 0.3]`, to assess the impact of adversarial attacks at different strengths. Naturally, a lower epsilon value corresponds to a less potent attack, but it also makes the attack less detectable.

To evaluate which features were most affected by the adversarial attacks, we defined a function to calculate the feature perturbations, which measures the difference between the original and adversarial samples.

4.3.1 FastGradientSignMethod - FGSM. As the first adversarial attack, we used the Fast Gradient Sign Method (FGSM), which works by perturbing the input data in the direction of the gradient of the loss function with respect to the input. The perturbation is scaled by a factor ϵ , which controls the magnitude of the attack. This method is simple yet effective in generating adversarial examples that can mislead the model.

The results obtained are shown in Figure 20. We can observe that even with low values of ϵ , the robust accuracy is significantly lower compared to the "clean validation" accuracy. As the ϵ value increases, the accuracy continues to decrease noticeably, ultimately converging to values around 9%.

Among the most perturbed features in we identified `Source.Port`, `min_seg_size_forward`, `hour`, `minute`, and `second`. This suggests that these features are more sensitive to adversarial perturbations, indicating they play a critical role in the model's decision-making process.

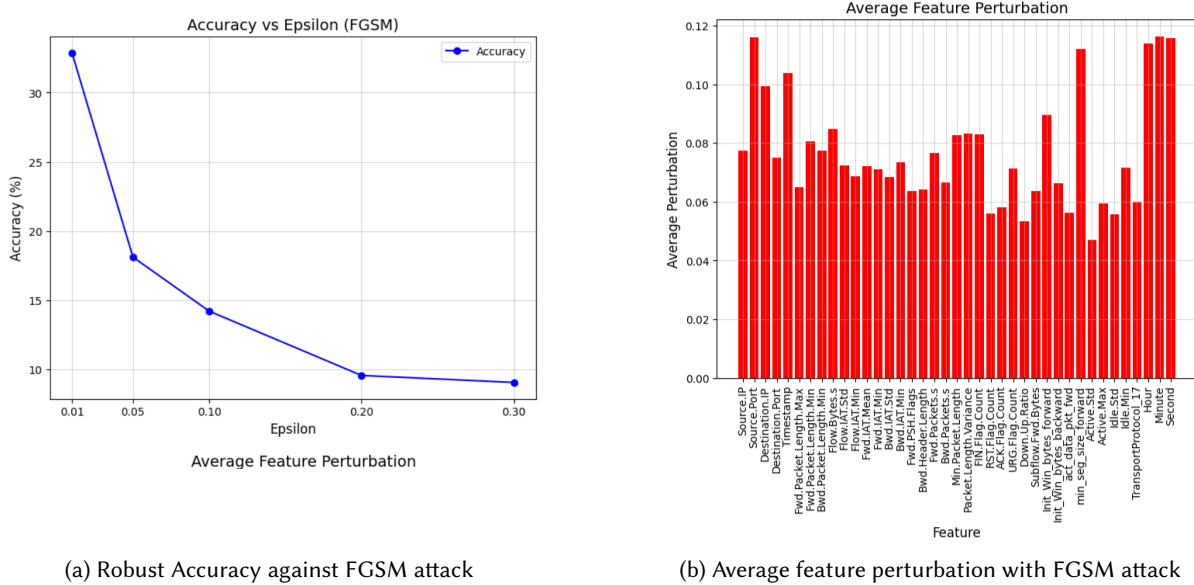


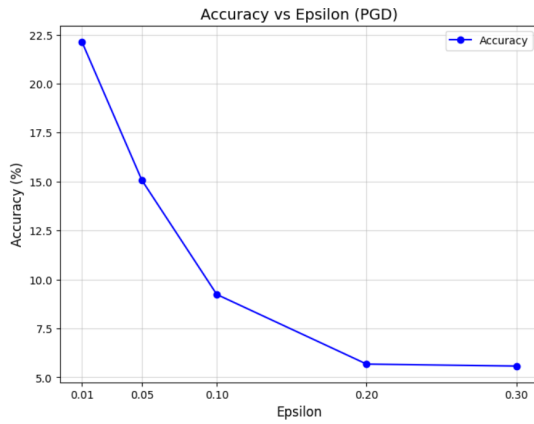
Fig. 19. Impact of FGSM attack

4.3.2 Projected Gradient Descent - PGD. As second attack we use the Projected Gradient Descent (PGD) [9], that is an iterative attack that extends the idea of FGSM by performing multiple small gradient steps: at each iteration, the perturbation is applied in the direction of the gradient and then projected back into a defined space (usually controlled by the epsilon parameter). This allows PGD to generate stronger adversarial examples than FGSM, as it explores the perturbation space more thoroughly and finds perturbations that are more effective in misleading the model.

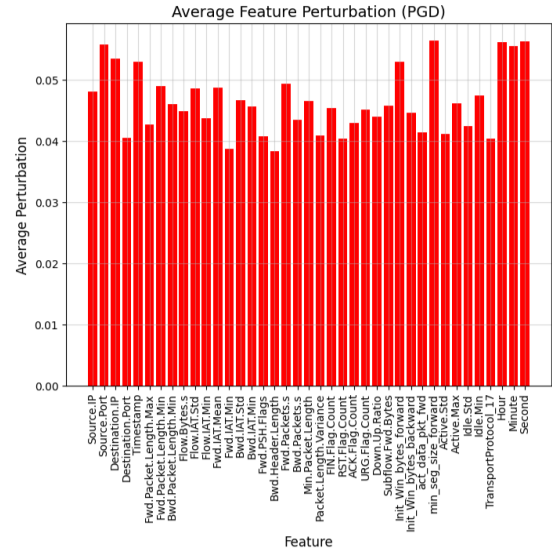
The PGD attack shows in Figure 20a a consistent trend of performance degradation. At $\epsilon = 0.01$, the robust accuracy is 19.52%, already lower than FGSM for the same epsilon value. As the attack strength increases, the robust accuracy progressively decreases, culminating in 3.27%. At $\epsilon = 0.01$, the robust shows a significantly more pronounced decline compared to FGSM for the same epsilon value (from 32.88% to 19.52%). This trend continues as the attack strength increases, with PGD consistently achieving lower robust accuracy across all epsilon values. These results underline that the PGD attack, being a more iterative and optimized approach, generally produces stronger adversarial perturbations compared to FGSM.

Regarding feature perturbations, in Figure 20b the PGD attack resulted in more consistent and significantly lower perturbation values compared to FGSM. The maximum feature perturbation observed with PGD was 0.055, which is notably lower than the 0.12 recorded for FGSM. This difference highlights the more refined nature of the PGD attack, which distributes the perturbation more evenly across features, rather than targeting specific ones more aggressively.

4.3.3 Comparison of attacks by ART and random noise. As might be expected, attacks made using ART library functions have a significantly greater impact on the Neural Network than simply adding random noise to the



(a) Robust Accuracy against PGD attack



(b) Average feature perturbation with PGD attack

Fig. 20. Impact of PGD attack

dataset. In fact, if via random noise with $\epsilon=1.0$ the accuracy drops to values on the order of 18% as seen in Figure 17, to get the same result via the FGSM or PGD attacks, an ϵ value of about 0.05 was sufficient, which is significantly lower.

Therefore, it is important to test the models using more sophisticated techniques so that more reliable robustness results can be obtained.

4.4 Countermeasure Exploration

Exploring various strategies to counteract adversarial attacks, one promising approach is adversarial training. In this technique, the model is trained on a mix of clean and adversarial samples to increase its robustness and improve its resistance to future attacks of this nature. To implement this strategy, we utilized a tool provided by the ART library, which facilitates the generation of adversarial samples during training and seamlessly integrates them into the learning process [10]. This approach aims to harden the model against both known and unknown adversarial threats, ultimately enhancing its security and reliability in real-world applications.

After training the model with 40% of adversarial examples in each training batch, generated by the attacks FGSM and PGD, and the remaining 60% consisting of clean samples, we proceeded to evaluate its performance.

The first evaluation on the clean test set resulted in an accuracy of 64.1%, slightly lower than the accuracy of the model trained exclusively on clean data.

Subsequently, new adversarial samples were generated for both FGSM and PGD attacks, and the performance of the newly trained model was evaluated. The results demonstrated a significantly improved robustness compared to the baseline model.

This trend persisted for all epsilon values, with the new model achieving a minimum accuracy of **22.5%** at $\epsilon = 0.30$, significantly outperforming the baseline model, which only reached **9.1%**. These results indicate that adversarial training effectively enhanced the model's robustness, even under stronger attack scenarios.

Similar improvements were observed with the PGD attack, as shown in Figure 21b. At $\epsilon = 0.01$, the accuracy rose from 22.1% to 38.3%.

However, as ϵ increased, the gap between the adversarially trained model and the baseline model narrowed, with the new model achieving a minimum accuracy of 12.8% at $\epsilon = 0.30$, compared to 5.6% for the baseline model. This suggests that while adversarial training significantly enhances robustness at lower attack strengths, its effectiveness diminishes as the attack intensity increases.

These results underscore the effectiveness of adversarial training in enhancing the model's robustness against both FGSM and PGD attacks. However, similar to FGSM, the model still struggles as the strength of the attack grows, particularly with larger epsilon values.

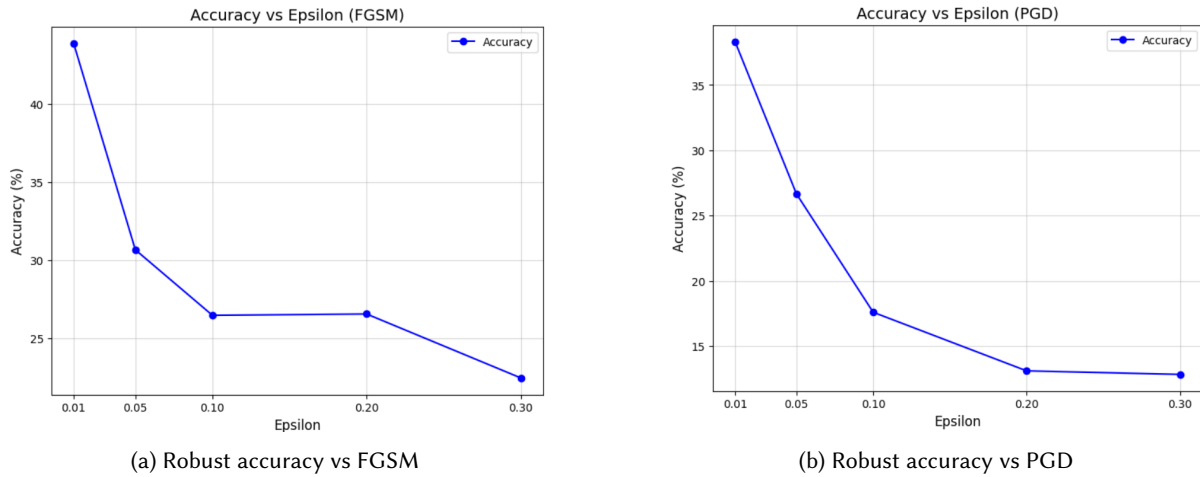


Fig. 21. Robust accuracy for the adversarially trained model

5 Conclusion

This project explored various machine learning techniques, as detailed in the sections on data acquisition and exploration, unsupervised data analysis, and supervised data analysis. Key stages included preprocessing, data visualization, classifier selection, and evaluation. These steps allowed for an in-depth analysis of the potential impacts of specific cyberattacks, not only on systems but also on machine learning models themselves.

The primary objective was to evaluate how machine learning models respond to attack scenarios, identify vulnerabilities, and propose potential mitigation strategies. In particular, adversarial training emerged as a promising solution to enhance model robustness against certain attacks.

For future developments, it would be valuable to consider employing more complex and advanced models to better simulate the performance of commercial models currently in use. Specifically, a more powerful neural network could be utilized, incorporating additional hidden layers and a greater number of neurons per layer. This approach would likely improve the model's representational capacity and robustness. However, these advancements were not feasible within the scope of this project due to hardware and time constraints. Additionally, exploring novel defense techniques and alternative methodologies could provide further insights into mitigating the effects of adversarial attacks, thereby strengthening the security and resilience of machine learning models.

References

- [1] Luca Vassio and Gabriele Ciravegna. Laboratory 7: Supervised learning. Course Material, 2024.
- [2] Luca Vassio and Gabriele Ciravegna. Laboratory 4: Data visualization. Course Material, 2024.
- [3] Luca Vassio and Gabriele Ciravegna. Laboratory 5: Dimensionality reduction. Course Material, 2024.
- [4] Luca Vassio and Gabriele Ciravegna. Laboratory 9: Unsupervised learning: Clustering. Course Material, 2024.
- [5] Luca Vassio and Gabriele Ciravegna. Laboratory 8: Supervised learning with model selection, validation and hyper-parameter. Course Material, 2024.
- [6] Luca Vassio and Gabriele Ciravegna. Laboratory 10: Neural networks. Course Material, 2024.
- [7] Adversarial Robustness Toolbox. Art documentation, 2024. URL: <https://adversarial-robustness-toolbox.readthedocs.io/en/latest/modules/attacks/evasion.html>.
- [8] Adversarial Robustness Toolbox. Getting started with pytorch example, 2024. URL: https://github.com/Trusted-AI/adversarial-robustness-toolbox/blob/main/examples/get_started_pytorch.py.
- [9] Adversarial Robustness Toolbox. Projected gradient descent implementation, 2024. URL: https://github.com/Trusted-AI/adversarial-robustness-toolbox/blob/main/art/attacks/evasion/projected_gradient_descent/projected_gradient_descent.py.
- [10] Adversarial Robustness Toolbox. Art documentation: art.defence.trainer, 2024. URL: <https://adversarial-robustness-toolbox.readthedocs.io/en/latest/modules/defences/trainer.html>.